Object-Oriented Programming — DAUST Fall 2026

Midterm Review (practice version) — Lectures 1 to 5 (excluding instanceof)

Duration: 1 h 50 min. **Total:** 50 questions × 2 points = 100 (35 multiple choice + 15 fill in the blank). **Passing grade: 74.**

Instructions: No notes, no devices. Multiple choice: exactly one correct answer — circle one letter; more than one circled letter scores 0. Fill in the blank: write one word or code token on the line; spelling of keywords and identifiers counts.

Review version: same format, same topics, same difficulty as the midterm, with different questions. Time yourself: 1 h 50 min.

Full name		**Student ID**		**Total**	/ 100

Part A — Why OOP Exists (Lecture 1)

1. According to Lecture 1, turning a tiny one-course, three-student program into classes would most likely:

A. Remove the shared state that makes such a small script unsafe
B. Be required, because TypeScript forbids top-level functions
C. Make it run faster, because methods are compiled in advance
D. Make it more complicated without solving a real problem

2. A developer adds a course and its capacity, but forgets the enrolment list:

```
let courseNames: string[] = ["OOP"];
let capacities: number[] = [30];
let enrolled: string[][] = [["Awa"]];
courseNames.push("Databases");
capacities.push(40);
console.log(enrolled[1]);
```

What is printed?

A. `undefined` **B.** `[]` **C.** `["Awa"]` **D.** A compile-time error

3. `enroll()` refuses duplicate students, but `adminEnroll()` pushes into `enrolledStudents` **without** that check. Which failure from Lecture 1 is this?

A. Scattered rules **B.** Indirection **C.** Shotgun change **D.** Over-engineering

4. According to Lecture 1, grouping data and functions is not the main idea of an object. The deeper idea is that:

A. The object stores its data in a faster structure than arrays
B. The object is responsible for keeping its own state correct
C. Every function in the program may update the object's fields
D. The object can be copied into many files of the program

5. Consider:

```
class Room {
  seats = 25;
}
const lab = new Room();
const hall = new Room();
hall.seats = 120;
console.log(lab.seats);
```

What is printed?

A. `120` **B.** `145` **C.** `25` **D.** `undefined`

Part B — Encapsulation and Responsibilities (Lecture 2)

6. Lecture 2 summarises encapsulation as: **state + rules = ?**

A. Responsibility **B.** Inheritance **C.** Abstraction **D.** Polymorphism

7. A caller lends a book like this:

```
if (book.isAvailable() && member.getLoanCount() < 3) {
  book.setAvailable(false);
  member.setLoanCount(member.getLoanCount() + 1);
}
```

Which statement best describes this code?

A. Tell: the caller delegates the decision to the book
B. Good encapsulation, because each object uses getters
C. Ask: the caller makes a decision the library objects should own
D. Polymorphism: each object chooses which version runs

8. Why does Lecture 2 say getters and setters are **not** the goal of encapsulation?

A. Getters and setters are rejected by the TypeScript compiler
B. Getters turn private fields back into public ones at runtime
C. If callers do the thinking, the rule still lives outside the object
D. Setters may only be called from inside a class constructor

9. Which item is **not** a reason given in Lecture 2 for OOP to earn its cost?

A. Rules governing changing state **B.** Multiple interacting parts
C. Meaningful responsibilities **D.** A wish to maximise the number of classes

10. Among the four pillars, which one is summarised as **expose a useful interface while hiding complexity**?

A. Polymorphism **B.** Inheritance **C.** Abstraction **D.** Encapsulation

Part C — TypeScript Fundamentals (Lecture 3)

11. What happens when this code is type-checked?

```
let year = 2026;
year = "2027";
```

A. It compiles: `year` was inferred as `any`
B. Compile-time error: `year` was inferred as `number`
C. It compiles, but the program throws an error at runtime
D. It compiles: a `let` variable may change its type

12. Consider:

```
function openRoom(code: string, seats: number = 25): string {
  return `${code}/${seats}`;
}
console.log(openRoom("B12"));
```

What is printed?

A. `B12/undefined` **B.** `B12/25`
C. `B12/0` **D.** A compile-time error: missing argument

13. To describe a contract such as `pay(amount: number): void` that classes will promise to satisfy, the course's practical rule recommends:

A. A `type` alias **B.** A union type **C.** A `readonly` field **D.** An `interface`

14. Given `interface CourseInfo { readonly code: string; title: string; }` and a value `c: CourseInfo`, which line does the compiler reject?

A. `c.title = "Databases";` **B.** `console.log(c.code);` **C.** `c.code = "CS202";` **D.** `const t = c.title;`

15. Consider:

```
class Account {
  protected balance = 0;
}
class SavingsAccount extends Account {
  addInterest(rate: number): void {
    this.balance += this.balance * rate;   // line 1
  }
}
const acc = new SavingsAccount();
acc.balance = 1000;                         // line 2
```

Which statement is true?

A. Line 1 is rejected; line 2 compiles **B.** Both lines are rejected
C. Both lines compile **D.** Line 1 compiles; line 2 is rejected

16. Consider:

```
class Wallet {
  private balance = 500;
}
const w = new Wallet();
console.log(JSON.stringify(w));
```

What is printed when the file is run with `node`?

A. `{"balance":500}` **B.** `{}`
C. `undefined` **D.** A compile-time error: `balance` is private

17. Node runs `.ts` files by stripping the types. Which declaration will it **refuse** to run?

A. `enum Color { Red, Green }` **B.** `private credits = 0;`
C. `let id: string | number;` **D.** `interface Printable { print(): void; }`

18. Consider:

```
class Club {
  private members: string[] = ["Awa", "Bob", "Fatou", "Moussa"];
  getMembers(): string[] { return this.members; }
  size(): number { return this.members.length; }
}
const club = new Club();
club.getMembers().pop();
console.log(club.size());
```

What is printed?

A. `4` **B.** `3`
C. A compile-time error: `members` is private **D.** A runtime error

Part D — Inheritance (Lecture 4)

19. DAUST creates `ExchangeStudent` and `DoctoralStudent` by copying `Student`. What is the main risk when a shared rule changes?

A. The compiler rejects classes with identical method bodies
B. The copies share one state, so their changes interfere
C. Every copy must be edited, and a forgotten copy stays wrong
D. The copies can no longer be stored in the same array

20. Consider:

```
class LabCourse extends Course {
  private room: string;
  constructor(code: string, room: string) {
    this.room = room;
    super(code);
  }
}
```

Does this compile?

A. No: `super` must be called before accessing `this`
B. Yes: statements in a constructor may come in any order
C. No: a subclass may not declare its own fields
D. Yes, but `super` resets `room` to `undefined`

21. `LabCourse extends Course`. During `new LabCourse("CS201", "B12")`, which statement is true?

A. The lab part is built first, then `super` builds the `Course` part
B. The `Course` part is built first via `super`, then the lab part; one object exists
C. Two objects are created, a `Course` and a `LabCourse`, linked together
D. Only the `LabCourse` constructor runs; `Course`'s is skipped

22. Consider:

```
class Student {
  private credits = 0;
  getCredits(): number { return this.credits; }
}
class ExchangeStudent extends Student {
  addBonus(): void {
    this.credits += 3;
  }
}
```

What does the compiler report?

A. Nothing: a subclass inherits full access to every field
B. Error: `credits` is private and only accessible within `Student`
C. Error: the subclass must declare its own `credits` field
D. Nothing, but `addBonus()` has no effect on the credits

23. Consider:

```
class Course {
  private code: string;
  constructor(code: string) { this.code = code; }
  describe(): string { return `Course ${this.code}`; }
}
class LabCourse extends Course {
  private room: string;
  constructor(code: string, room: string) { super(code); this.room = room; }
  override describe(): string { return `${super.describe()} in ${this.room}`; }
}
console.log(new LabCourse("CS201", "B12").describe());
```

What is printed?

A. `CS201 in B12`
B. `Course CS201`
C. `Course CS201 in B12`
D. A compile-time error: `code` is private

24. Consider:

```
class Student {
  private credits = 0;
  protected getCreditLimit(): number { return 21; }
  addCredits(n: number): boolean {
    if (this.credits + n > this.getCreditLimit()) return false;
    this.credits += n;
    return true;
  }
}
class ExchangeStudent extends Student {
  protected override getCreditLimit(): number { return 15; }
}
```

With `const e = new ExchangeStudent();`, what do `e.addCredits(12)` and then `e.addCredits(3)` return?

A. `true`, then `false`
B. `false`, then `true`
C. `true`, then `true`
D. `false`, then `false`

25. `class Library extends BookList` reuses `add()`, `has()` and `removeAll()`. `addBook()` refuses duplicate books. What is the problem?

- **A.** It does not compile: `Library` must redeclare the `books` field
- **B.** `addBook()` cannot call `has()`, because inherited methods are private
- **C.** Each `Library` object holds two separate lists, wasting memory
- **D.** Callers can also call `add()` and `removeAll()`, going around `addBook()`

26. Which declaration passes the **is-a** test?

- **A.** `class GraduateStudent extends Student`
- **B.** `class Course extends StudentList`
- **C.** `class Building extends Classroom`
- **D.** `class Transcript extends Student`

27. Mamadou is a teaching assistant for one year only, then stops. According to Lecture 4, which test does `TeachingAssistant extends Student` fail?

- **A.** The substitution test
- **B.** The override test
- **C.** The stability test
- **D.** It passes all three tests

Part E — Polymorphism and Abstract Classes (Lecture 5)

28. `getBorrowingLimit()` uses an `if`/`else if` chain on `student.type` and ends with `return 5;`. A `"visiting"` student type is added, but this function is forgotten. What happens?

- **A.** The compiler reports that the `visiting` branch is missing
- **B.** It silently returns `5` for a visiting student; no error is reported
- **C.** The program throws an error as soon as the function runs
- **D.** It returns the limit of the closest matching student type

29. Consider:

```
abstract class Student { abstract calculateTuition(): number; }
class DoctoralStudent extends Student {
  override calculateTuition(): number { return 200000; }
}
class GraduateStudent extends Student {
  override calculateTuition(): number { return 150000; }
}
const group: Student[] = [new DoctoralStudent(), new GraduateStudent(), new DoctoralStudent()];
let total = 0;
for (const s of group) total += s.calculateTuition();
console.log(total);
```

What is printed?

- **A.** `200000`
- **B.** `0`
- **C.** A compile-time error: `Student` is abstract
- **D.** `550000`

30. Consider:

```
class Student {
  describe(): string { return "student"; }
}
class ExchangeStudent extends Student {
  override describe(): string { return "exchange"; }
  getHomeUniversity(): string { return "Laval"; }
}
const s: Student = new ExchangeStudent();
console.log(s.getHomeUniversity());   // line 1
console.log(s.describe());            // line 2
```

Which statement is true?

- **A.** Line 1 is a compile-time error; line 2 prints `exchange`
- **B.** Both lines compile; they print `Laval` and `exchange`
- **C.** Line 1 is a compile-time error; line 2 prints `student`
- **D.** Both lines compile; they print `Laval` and `student`

31. In the Open/Closed Principle applied to DAUST, **closed for modification** means:

- **A.** `totalTuition()` does not change when a new kind of student is added
- **B.** `Student` can no longer be extended by new subclasses
- **C.** the fields of `Student` cannot change after construction
- **D.** no subclass may add methods beyond those of `Student`

32. Consider:

```
abstract class Student {
  abstract calculateTuition(): number;
}
const s = new Student();
```

What happens?

- **A.** It compiles; `s` is a plain `Student` with no tuition rule
- **B.** It compiles, but throws when `calculateTuition()` is called
- **C.** Compile-time error: `Student` must declare a constructor
- **D.** Compile-time error: cannot create an instance of an abstract class

33. Which of the following can an abstract class **not** do?

A. Declare `private` fields
B. Define a constructor
C. Contain implemented methods
D. Be instantiated directly with `new`

34. In the Template Method pattern, which part do the **subclasses** provide?

A. The steps that vary, such as `getCreditLimit()`
B. The sequence of steps, such as the order of the checks
C. The shared validation, such as the whole-number rule
D. The constructor of the base class, via `super`

35. A library must compute late fees for `Book` and `Laptop` loans. The two classes are unrelated and share no code or state. Which design fits Lecture 5's practical rule?

A. `Laptop extends Book` to reuse the fee method
B. Both classes implement a shared `Chargeable` interface
C. A `kind` field checked by an `if` chain in one function
D. `Book extends Laptop`, since both can be borrowed

Part F — Fill in the Blanks

36. The rule *a student cannot enroll twice in the same course* must always hold. Such a rule is called an _____.

37. In Lecture 1's list of failures, data that can be modified from many places is called shared _____ state.

38. Calling `course.enroll(student)` instead of checking the capacity in the caller follows the principle _____, Don't Ask.

39. Complete the property so that `code` can be read but never reassigned.

```
interface CourseInfo {
  _____ code: string;
  title: string;
}
```

40. Complete the declaration so that the variable can hold a `Student` or `null`.

```
let selectedStudent: Student _____ null = null;
```

41. Complete `Course.ts` so that it can use the `Student` class from another file.

```
// Course.ts
_____ { Student } from "./Student.ts";

export class Course { /* ... */ }
```

42. Complete the `tsconfig.json` setting that checks only and does not generate `.js` files.

```
{
  "compilerOptions": {
    "_____": true   // check only; do not generate .js files
  }
}
```

43. Complete the cast that switches the checks off (and should never be used to bypass `private`).

```
(student as _____).credits = 1000;
```

44. Complete the override so that it runs the parent's version of `describe()` on the same object.

```
override describe(): string {
  return `${_____.describe()}, thesis: ${this.thesis}`;
}
```

45. Complete the keyword that tells the compiler this method replaces the parent's version.

```
class ExchangeStudent extends Student {
  protected _____ getCreditLimit(): number {
    return 15;
  }
}
```

46. Lecture 4's principle: favour _____ over inheritance.

47. Open/Closed Principle: software should be open for _____, but closed for modification.

48. Complete the declaration of a method signature with no body that every concrete subclass must implement.

```
abstract class Student {
  _____ calculateTuition(): number;
}
```

49. Complete the keyword that declares a contract listing members, with no implementation and no state.

```
_____ Billable {
  calculateCharge(): number;
}
```

50. In `const s: Student = new ExchangeStudent(...)`, the _____ type decides which methods may be called; the actual type decides which version runs.

— End of paper —